

Actividad Evaluativa 2 - Informe individual

Buscador y filtro de catálogo en tiempo real

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación III

COMISIÓN: A

PROFESOR: Sicardi, Dante

ESTUDIANTE: Tarnoski, Santiago

FECHA: 29-09-2026

REPOSITORIO: <https://github.com/santiagotarnoski/gym-control-ae1>



INTRODUCCIÓN

Gym Control es un sitio de consulta de planes de gimnasio desarrollado en la Actividad Evaluativa 1. Esta Actividad Evaluativa 2 amplía el catálogo en tarjetas con un buscador y un filtro por duración. La solución conserva los cuatro planes existentes y permite localizar una opción antes de acceder al formulario de selección.

Desarrollé la tarjeta 3, Buscador y filtro de catálogo en tiempo real. Registré los eventos con `addEventListener`, cargué el catálogo mediante `fetch` y `async/await`, filtré los datos en memoria con `Array.filter` y actualicé el DOM con los resultados. Para fundamentar mi aporte, apliqué SOHDM al escenario de exploración y búsqueda.

Presento mi aporte individual a la tarjeta 3, su fundamentación con SOHDM, las decisiones técnicas que tomé y las evidencias que reuní. Complementa el informe grupal, en el que documentamos las tres tarjetas del equipo. El alcance es académico: los datos son de ejemplo, no hay persistencia ni contratación real.

Responsabilidad individual y organización del equipo

Me hice cargo del buscador y filtro del catálogo: cargué el JSON, registré los eventos, filtré los planes, presenté los resultados y conecté la selección con el formulario. También elaboré el escenario E01 y su mapa navegacional con SOHDM.

Mis compañeros Franco Olexyn y Santino Martins desarrollaron, respectivamente, la tarjeta 4 de acceso de socios con VSDM y la tarjeta 11 de reclamos y consultas con una fundamentación conceptual de RNA. En este informe describo mi aporte, la tarjeta 3, y explico cómo se integra con los módulos de ellos.

Alcance y trazabilidad

Requisito	Evidencia en la implementación
Continuidad con AE1	Los cuatro planes, precios, páginas y estilos de Gym Control permanecen.
Eventos desacoplados	<code>input</code> , <code>change</code> y <code>click</code> se registran en <code>js/app.js</code> con <code>addEventListener</code> .
HTTP y asincronía	<code>fetch</code> y <code>async/await</code> cargan <code>data/planes.json</code> desde un servidor local.
Búsqueda dinámica	<code>Array.filter</code> combina texto y duración; el DOM presenta tarjetas y contador.
Metodología SOHDM	Escenario E01, objetos y mapa navegacional de consulta y selección.

DESARROLLO



1 Descripción del módulo

Incorporé el módulo en listado_box.html, accesible desde la portada y desde el listado en tabla. Al iniciar la página, mi código consulta por HTTP data/planes.json antes de mostrar la búsqueda. Cada registro contiene el identificador, nombre, duración, precio, categoría, descripción y marca de destacado. Conservé los precios del proyecto original: Mensual \$25.000, Trimestral \$67.500, Semestral \$120.000 y Anual \$216.000.

Implementé un campo de texto que busca en el nombre, la descripción y la categoría, junto con un selector de 1, 3, 6 o 12 meses. Combiné ambos criterios para que la lista se actualice al cambiar cualquiera de ellos. También incorporé la limpieza de filtros y la selección del plan. Comprobé que renovacion encuentra renovación sin tilde y que mensual también coincide con el Plan Anual por la frase valor mensual de su descripción.

2 Organización técnica

Componente	Responsabilidad
listado_box.html	Presenta controles, estado de carga, contenedor y mensaje sin resultados.
data/planes.json	Almacena los cuatro planes de ejemplo de la AE1.
js/app.js	Carga, valida, filtra, crea tarjetas y conserva el plan al navegar.
css/styles.css	Mantiene el diseño y adapta los controles a la vista móvil.
comprar.html	Recibe el identificador del plan y conserva las funciones previas.

Implementé la carga con fetch("data/planes.json") y await. Verifico response.ok y valido el formato antes de habilitar los filtros. Si se produce un error HTTP, de red o de datos, muestro un mensaje y el botón Reintentar. Diferencio un catálogo vacío de una búsqueda sin coincidencias. Cuando cambian los criterios, aplico Array.filter sobre los registros cargados y evito repetir la petición [2, 3].

Construyo las tarjetas con createElement y textContent y las inserto con replaceChildren, para tratar el contenido JSON como texto. Integré la selección mediante el parámetro plan: al abrir el formulario, mi cambio preselecciona el radio del plan y dispara su evento change para actualizar el subtotal. Conservé la compatibilidad con las funciones existentes de subtotal y cupón.

3 Escenario E01 de exploración y búsqueda

Actor principal: visitante o socio interesado en comparar planes. Precondición: el sitio se sirve mediante HTTP y el catálogo puede cargarse. Objetivo: encontrar un plan compatible con los criterios del usuario y continuar a su selección.



Paso	Acción y respuesta esperada
1	Abrir Planes en cards; el sistema carga y presenta cuatro tarjetas.
2	Escribir progreso; aparece el Plan Semestral por su categoría.
3	Elegir 6 meses; permanece el resultado compatible.
4	Elegir el plan; se abre comprar.html?plan=semestral.
5	Consultar el formulario; el plan aparece marcado y el subtotal se calcula.

Flujos alternativos: si los filtros no encuentran registros, se muestra cero resultados y se permite modificar o limpiar la consulta. Si falla la carga, se bloquean los controles y se ofrece reintento. Si el JSON está vacío, se indica que no hay planes disponibles. La postcondición del flujo principal es una selección visual de plan; no se asigna una membresía.

4 Fundamentación de SOHDM

SOHDM (Scenario Based Object Oriented Hypermedia Design Methodology) usa escenarios para identificar requisitos de sistemas hipermedia y emplea vistas de objetos para definir unidades navegacionales. Lee, Lee y Yoo describen seis fases: análisis del dominio, modelado de objetos, diseño de vistas, diseño de navegación, diseño de implementación y construcción [1].

Fase	Aplicación en Gym Control
Análisis del dominio	Actor, objetivo, flujo principal y alternativas del escenario E01.
Modelado de objetos	Plan, Consulta y Catálogo como objetos conceptuales.
Diseño de vistas	Barra de filtros, tarjetas, contador y estados de error o vacío.
Diseño de navegación	Accesos desde Inicio y tabla; salida a detalle o selección.
Diseño de implementación	JSON, funciones JavaScript, estructura HTML y estilos.
Construcción	Integración del código y pruebas de navegador con servidor HTTP.

Modelé Plan como el objeto con los atributos que busco y presento, Consulta como el texto normalizado y la duración, y Catálogo como el conjunto cargado y filtrado. En las vistas mostré la información necesaria para explorar y decidir. Apliqué las ideas de SOHDM al diseño; no implementé un motor formal de esa metodología.

5 Mapa navegacional

Derivé el mapa navegacional del escenario E01. Conservé el recorrido Inicio → Listado en tabla o Catálogo en tarjetas. Representé búsqueda y resultados como estados de listado_box.html y no como páginas independientes. Desde cada tarjeta, el usuario puede continuar a la selección mediante el ID del plan. Mantuve la ficha producto.html para el Plan Mensual.

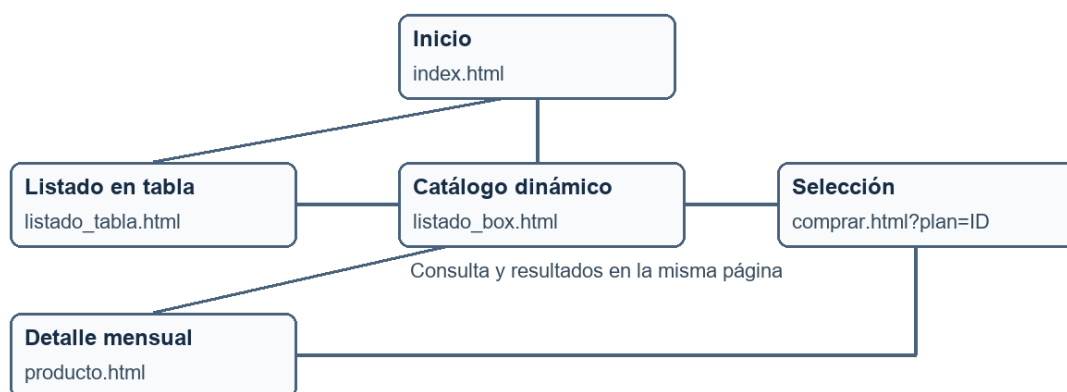


Figura 1 Mapa navegacional del escenario de exploración y búsqueda

Incorporé enlaces para volver a la tabla, cambiar los filtros o limpiar la consulta. Si falla la carga, el botón Reintentar vuelve a ejecutar la petición desde el mismo nodo. Guardé el diagrama editable en docs/mapa-navegacional-tarjeta3.mmd.

6 Interacción y estados de la interfaz

Vinculé input del campo de búsqueda y change del selector de duración con filtrarPlanes(). Configuré Limpiar filtros para restaurar los controles y devolver el foco al texto; Reintentar vuelve a llamar a cargarCatalogo(). Registré estos eventos con addEventListener en js/app.js y evité los atributos onclick y onsubmit en el HTML [4].

Agregué un contador para informar cuántos planes se muestran de los cuatro disponibles. Usé role="status" y aria-live="polite" para comunicar resultados, y aria-busy para señalar la carga. Mantengo los controles deshabilitados mientras la respuesta no sea válida. Adapté los controles y las tarjetas a una columna en teléfonos.

7 Verificación

Ejecuté 36 comprobaciones en Google Chrome con un servidor HTTP local. Dejé el registro en docs/pruebas-tarjeta3.json y el procedimiento reproducible en tests/catalogo.cjs. Comprobé la carga de las cuatro tarjetas, búsquedas con tildes y mayúsculas, filtros combinados, cero resultados, limpieza, selección de plan, compatibilidad con subtotal y cupón, vista móvil y errores HTTP, de red o de JSON.



Caso representativo	Resultado observado
Búsqueda por progreso y 6 meses	Solo se presenta el Plan Semestral.
Búsqueda incompatible	Se informa 0 de 4 planes y se ofrece limpiar.
Plan Trimestral elegido	comprar.html preselecciona el plan y muestra \$67.500.
Error HTTP 404	Los filtros se bloquean; el reintento recupera el catálogo.
Vista de 390 píxeles	No existe desbordamiento horizontal.

Obtuve 36 verificaciones correctas. Comprobé que el JSON se inserte como texto y que el recorrido normal no produzca errores JavaScript. Incluí las capturas de pantalla en los anexos.

8 Decisiones técnicas y relación con el trabajo grupal

Decidí separar la carga del catálogo de la búsqueda. Hago la petición HTTP al iniciar la página o al reintentar una carga fallida; cada cambio de texto o duración usa el arreglo que ya está en memoria. Así evito peticiones repetidas y actualizo los resultados inmediatamente.

Conservé la selección al navegar mediante el identificador del plan en la URL. Integré ese recorrido con el formulario existente y su cálculo de subtotal. Distinguí los estados de carga, error, catálogo vacío y ausencia de coincidencias para orientar al usuario y evitar que filtre datos que todavía no están disponibles.

En la versión grupal, los bloques de las tarjetas 3, 4 y 11 conviven en js/app.js. El catálogo sigue siendo público para el visitante; el acceso del socio y el formulario de reclamos amplían el recorrido sin sustituir la búsqueda. La comprobación de cada módulo se documenta por separado, por lo que las 35 verificaciones de acceso y las 12 de reclamos no se presentan aquí como resultados de la tarjeta 3.

En mi trabajo relacioné los requisitos del escenario con eventos y estados concretos de una página web. Apliqué asincronía, validación de respuestas HTTP, operaciones sobre arreglos y actualización del DOM. Como mejora futura, propongo conectar el catálogo con un servicio persistente y revisar el contrato de datos y sus rutas de error.



CONCLUSIONES FINALES

En esta Actividad Evaluativa 2 incorporé a Gym Control el buscador y filtro de la tarjeta 3, complementario al subtotal y la validación del cupón desarrollados previamente. Cumplí la tarjeta 3 al registrar eventos desacoplados, cargar el catálogo de forma asíncrona mediante HTTP, filtrar los datos con `Array.filter` en memoria y actualizar el DOM de inmediato.

A partir del escenario E01 definí los campos de consulta, los estados de resultados y los accesos entre el catálogo y el formulario. Diseñé el mapa navegacional SOHDM para relacionar esa interacción con las pantallas existentes y dar continuidad al proyecto. Con las pruebas comprobé los casos esperados y las rutas de error del prototipo.

Dejé en el catálogo JSON datos comerciales de ejemplo. Para operar membresías reales, propongo incorporar un servicio de servidor, persistencia y validación de las operaciones. Mi entrega cubre la consulta, exploración y selección visual de planes.

BIBLIOGRAFÍA

- [1] Lee, H., Lee, C., & Yoo, C. (1999). A scenario-based object-oriented hypermedia design methodology. *Information & Management*, 36(3), 121-138.
[https://doi.org/10.1016/S0378-7206\(99\)00011-7](https://doi.org/10.1016/S0378-7206(99)00011-7)
- [2] MDN Web Docs. (s. f.). Using the Fetch API.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- [3] MDN Web Docs. (s. f.). `Array.prototype.filter()`.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- [4] MDN Web Docs. (s. f.). `EventTarget.addEventListener()`.
<https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener>
- [5] Tarnoski, S., Martins, S., & Olexyn, F. (2026). Gym Control: análisis y diseño de la Actividad Evaluativa 1. Repositorio del proyecto. <https://github.com/santiagotarnoski/gym-control-ae1>
- [6] Martins, S., Olexyn, F., & Tarnoski, S. (2026). Actividad Evaluativa 2: integración JS y metodologías web en Gym Control [Informe grupal]. Repositorio del proyecto.
<https://github.com/santiagotarnoski/gym-control-ae1>



ANEXOS

Anexo A Evidencia de búsqueda

Buscá el plan que se adapte a vos

Buscar por nombre, descripción o categoría

compromiso



Duración del plan

12 meses

1 de 4 planes encontrados.

Compromiso

Plan Anual
12 meses

\$ 216.000
/ total

La opción de mayor duración y mejor valor mensual.

Elegir plan

Figura 2 Búsqueda por categoría y duración con un resultado

Anexo B Estado sin resultados

Buscá el plan que se adapte a vos

Buscar por nombre, descripción o categoría

compromiso



Duración del plan

1 mes

0 de 4 planes encontrados.

No encontramos planes con esos filtros. Probá otra búsqueda o limpiá los filtros.

Figura 3 Filtros incompatibles y mensaje al usuario